

ui.range 全面详解（基于 NiceGUI 文档）

ui.range 是 NiceGUI 中用于实现**数值范围选择**的核心交互组件，基于 Quasar 的 QRange 组件实现。其核心价值在于通过双滑块直观选择连续或离散的数值区间（如 `[min_val, max_val]`），支持自定义区间上下界、步长、事件节流和样式定制，适用于价格筛选、时间范围选择、数值区间配置等需要精准划定范围的场景。与 `ui.slider`（单滑块选单个值）相比，`ui.range` 专注于**区间选择**，交互逻辑和参数设计更贴合范围筛选需求。以下从核心特性、基础用法、高级功能等维度展开全面解析。

一、核心基础

1. 组件本质与核心特性

- 底层依赖：基于 Quasar 的 QRange 组件，继承其双滑块交互逻辑、数值约束机制和样式体系，确保跨设备交互一致性。
- 核心功能：支持设置区间整体上下界（`min/max`）、滑动步长（`step`），实时反馈当前选择的数值区间（`value` 为二元列表 `[start, end]`），支持事件节流和禁用状态。
- 交互特性：
 - 双滑块独立拖动：左侧滑块控制区间起始值，右侧滑块控制区间结束值；
 - 边界约束：起始值 $\leq$ 结束值，拖动时自动限制滑块位置（避免交叉）；
 - 区间联动：拖动滑块时实时更新区间值，释放滑块时触发 `on_change` 回调（默认行为）。

2. 初始化参数（核心配置）

参数名	类型	说明	默认值
min	float / int	区间整体下界（所有选择的起始值和结束值均 $\geq$ 此值）	0
max	float / int	区间整体上界（所有选择的起始值和结束值均 $\leq$ 此值）	100
step	float / int	滑块滑动步长（每次拖动的数值增量，0 表示连续数值）	1
value	list[float/int, float/int]	初始区间值（格式 <code>[start, end]</code> ，需满足 <code>min $\leq$ start $\leq$ end $\leq$ max</code> ）	中间区间（ <code>[(min+max)/4, 3*(min+max)/4]</code> ）
on_change	Callable	滑块释放时触发的回调函数（ <code>e.value</code> 为当前区间 <code>[start, end]</code> ）	-
color	str / None	滑块和选中区间的颜色（支持 Quasar 颜色、Tailwind 颜色、CSS 颜色）	"primary"
disabled	bool	是否禁用组件（禁用后滑块灰度显示，不可拖动）	False

参数名	类型	说明	默认值
label	str	组件标签（显示在滑块上方）	None
hint	str	提示文本（显示在滑块下方）	None

二、基础使用示例

1. 最简用法（区间展示联动）

展示基础区间配置、初始值，以及与标签的实时联动，直观呈现当前选择的数值范围：

```
from nicegui import ui

# 基础区间选择器（0-100，初始区间 [25, 75]）
range_slider = ui.range(min=0, max=100, value=[25, 75])

# 实时显示当前区间（格式化输出）
ui.label().bind_text_from(
    range_slider, 'value',
    lambda x: f'当前选择区间: {x[0]} - {x[1]}'
)

ui.run()
```

效果：

- 滑块分为左右两个，分别控制区间的起始值和结束值；
- 拖动任意滑块时，下方标签实时更新区间范围；
- 释放滑块时触发 `on_change` 回调（若配置）。

2. 禁用与启用控制

通过按钮控制组件的禁用状态，适配权限控制或流程锁定场景：

```
from nicegui import ui

range_slider = ui.range(min=0, max=100, value=[30, 80], label='价格筛选', hint='拖动滑块选择价格范围')

# 禁用/启用按钮组
with ui.row().classes('mt-2 gap-2'):
    ui.button('禁用筛选', on_click=range_slider.disable)
    ui.button('启用筛选', on_click=range_slider.enable)

ui.run()
```

效果：

- 点击“禁用筛选”后，滑块灰度显示，不可拖动，标签和提示文本也同步灰度；

- 点击“启用筛选”后，组件恢复正常交互。

三、核心功能与进阶用法

1. 事件节流与实时监听

通过 `throttle` 控制事件触发频率，避免拖动时高频回调，同时支持监听实时变化（拖动过程中触发）：

```
from nicegui import ui

with ui.column().classes('gap-4'):
    # 1. 默认行为：滑块释放时触发 on_change（无节流）
    ui.label('默认：释放滑块触发回调')
    range1 = ui.range(min=0, max=100, value=[20, 80])
    range1.on_change(lambda e: ui.notify(f'区间确认: {e.value[0]} - {e.value[1]}'))

    # 2. 实时监听：拖动过程中触发（节流 0.5 秒，避免高频触发）
    ui.label('实时监听：拖动过程中触发（节流 0.5s）').classes('mt-4')
    range2 = ui.range(min=0, max=100, value=[30, 70])
    # 订阅 update:model-value 事件实现实时监听
    range2.on(
        'update:model-value',
        lambda e: ui.notify(f'实时区间: {e.args[0][0]} - {e.args[0][1]}'),
        throttle=0.5 # 每 0.5 秒最多触发一次
    )

ui.run()
```

效果：

- 第一个区间：仅在释放滑块时弹出通知，适合需要用户确认最终区间的场景；
- 第二个区间：拖动过程中每 0.5 秒弹出一通知，适合实时反馈（如筛选结果即时更新）。

2. 自定义区间范围与步长

设置非默认区间上下界、步长，支持整数、小数区间选择，适配不同精度需求：

```
from nicegui import ui

with ui.column().classes('gap-4'):
    # 1. 整数区间（10-100，步长 5，初始 [30, 70]）
    ui.label('整数区间（步长 5）')
    int_range = ui.range(min=10, max=100, step=5, value=[30, 70])
    ui.label().bind_text_from(int_range, 'value', lambda x: f'区间: {x[0]} - {x[1]}')

    # 2. 小数区间（0-1，步长 0.05，初始 [0.2, 0.8]）
    ui.label('小数区间（步长 0.05）').classes('mt-2')
    float_range = ui.range(min=0, max=1, step=0.05, value=[0.2, 0.8])
    ui.label().bind_text_from(float_range, 'value', lambda x: f'区间: {x[0]:.2f} - {x[1]:.2f}')
```

```
ui.run()
```

效果:

- 整数区间滑块仅能停在 10、15、20... 等 5 的倍数位置;
- 小数区间支持 0.00、0.05、0.10... 等精度, 标签格式化显示两位小数。

3. 数据绑定 (双向同步)

通过 `bind_value` 实现组件与数据对象的双向绑定, 数据变化时组件自动更新, 反之亦然:

```
from nicegui import ui

class FilterConfig:
    def __init__(self):
        self.price_range = [50, 150] # 初始价格区间
        self.score_range = [3.5, 5.0] # 初始评分区间

config = FilterConfig()

with ui.column().classes('gap-4'):
    # 价格区间绑定
    ui.range(
        min=0, max=200,
        label='价格筛选',
        hint='选择商品价格范围',
        color='green'
    ).bind_value(config, 'price_range')

    # 评分区间绑定
    ui.range(
        min=0, max=5,
        step=0.1,
        label='评分筛选',
        hint='选择商品评分范围',
        color='orange'
    ).bind_value(config, 'score_range')

    # 显示绑定数据的实时状态
    ui.label().bind_text_from(
        config, 'price_range',
        lambda x: f'当前价格区间: {x[0]} - {x[1]} 元'
    )
    ui.label().bind_text_from(
        config, 'score_range',
        lambda x: f'当前评分区间: {x[0]:.1f} - {x[1]:.1f} 分'
    )

    # 手动修改数据对象, 组件自动同步
    ui.button('重置筛选条件', on_click=lambda: (
        setattr(config, 'price_range', [50, 150]),
        setattr(config, 'score_range', [3.5, 5.0])
    ))
```

```
)).classes('mt-2')

ui.run()
```

效果：

- 拖动滑块修改区间，`config` 对象对应的属性自动同步；
- 点击“重置筛选条件”按钮修改数据对象，两个区间组件自动恢复初始值，实现双向绑定同步。

4. 样式定制（颜色、尺寸、标签）

通过 `props`、`classes` 和 `style` 定制组件的颜色、尺寸、滑块样式等，适配界面设计需求：

```
from nicegui import ui

with ui.column().classes('gap-4'):
    # 1. 自定义颜色（红色）+ 始终显示区间标签
    ui.range(min=0, max=100, value=[20, 80], color='red').props('label-always')

    # 2. 紧凑模式+大尺寸
    ui.range(min=0, max=100, value=[30, 70], dense=True).props('size-lg')

    # 3. 隐藏刻度+自定义宽度
    ui.range(min=0, max=100, value=[40, 60]).props('no-ticks').classes('w-80')

    # 4. 自定义滑块和轨道样式（通过 CSS 变量）
    ui.range(min=0, max=100, value=[10, 90]).style('''
        --q-range-thumb-color: #667eea; /* 滑块颜色 */
        --q-range-track-color: #e2e8f0; /* 未选中轨道颜色 */
        --q-range-selected-track-color: #c5ceeb; /* 选中区间轨道颜色 */
        --q-range-thumb-size: 16px; /* 滑块大小 */
    ''')

ui.run()
```

效果：

- 四个组件分别展示不同样式，支持颜色、尺寸、刻度显示、滑块 / 轨道样式的深度定制；
- `label-always` 属性让区间值始终显示在滑块上方，无需 hover。

5. 动态修改区间配置

通过 `set_min`、`set_max`、`set_step`、`set_value` 等方法动态修改组件配置，适配动态场景（如根据用户角色调整筛选范围）：

```
from nicegui import ui

# 初始区间配置（0-100，步长 1）
range_slider = ui.range(min=0, max=100, step=1, value=[20, 80], label='动态区间示例')

# 动态修改配置的按钮组
```

```
with ui.row().classes('mt-2 gap-2'):
    # 扩大区间范围 (0-200)
    ui.button('扩大范围', on_click=lambda: range_slider.set_max(200))
    # 缩小区间范围 (50-150)
    ui.button('缩小范围', on_click=lambda: (
        range_slider.set_min(50),
        range_slider.set_max(150),
        range_slider.set_value([60, 140]) # 同步调整初始值, 避免超出新范围
    ))
    # 调整步长为 10
    ui.button('步长 10', on_click=lambda: range_slider.set_step(10))
    # 重置配置
    ui.button('重置', on_click=lambda: (
        range_slider.set_min(0),
        range_slider.set_max(100),
        range_slider.set_step(1),
        range_slider.set_value([20, 80])
    ))

ui.run()
```

效果：

- 点击按钮后，组件的区间范围、步长、初始值动态更新，无需刷新页面；
- 修改范围时，若原区间值超出新范围，会自动被约束到新的边界（如缩小到 50-150 时，原 [20,80] 自动调整为 [50,80]）。

四、核心属性与方法

1. 常用属性

属性名	类型	说明
classes	Classes[Self]	CSS 类（支持 Tailwind/Quasar 样式，作用于组件容器）
disabled	BindableProperty	是否禁用（可绑定数据动态控制）
html_id	str	HTML 元素 ID（2.16.0+ 版本支持）
min	float / int	区间整体下界（可动态修改）
max	float / int	区间整体上界（可动态修改）
step	float / int	滑动步长（可动态修改）
value	BindableProperty	当前区间（二元列表 <code>[start, end]</code> ，可绑定数据）
visible	BindableProperty	是否可见（可绑定数据）
color	str	滑块和选中区间的颜色（可动态修改）
label	str	组件标签（可动态修改）
hint	str	提示文本（可动态修改）

属性名	类型	说明
props	Props[Self]	Quasar 特性属性 (如 <code>label-always</code> 、 <code>no-ticks</code>)

2. 关键方法

(1) 状态控制

- `disable()`: 禁用组件 (不可拖动, 视觉灰度)
- `enable()`: 启用组件
- `set_disabled(value: bool)`: 设置禁用状态 (`True` 禁用, `False` 启用)
- `set_visibility(visible: bool)`: 设置组件可见性

(2) 配置修改

- `set_value(value: list[float/int, float/int])`: 动态设置区间值 (需满足 $\text{min} \leq \text{start} \leq \text{end} \leq \text{max}$, 触发 `on_change` 事件)
- `set_min(min: float / int)`: 动态修改区间整体下界 (若原起始值 < 新下界, 自动调整起始值为新下界)
- `set_max(max: float / int)`: 动态修改区间整体上界 (若原结束值 > 新上界, 自动调整结束值为新上界)
- `set_step(step: float / int)`: 动态修改滑动步长 (`step=0` 表示连续数值)
- `set_color(color: str)`: 动态修改滑块和选中区间的颜色
- `update()`: 修改属性后, 调用此方法刷新界面 (绑定数据时无需手动调用)

(3) 事件与绑定

- `on_change(callback)`: 绑定滑块释放时的事件 (`e.value` 为当前区间 `[start, end]`)
- `on(type: str, handler)`: 订阅任意 DOM 事件 (如 `update:model-value` 监听实时变化)
- `bind_value(target_object, target_name)`: 双向绑定区间值到目标对象的属性 (目标属性需为二元列表)
- `bind_min_from(target_object, target_name)`: 单向绑定区间下界从目标对象
- `bind_max_from(target_object, target_name)`: 单向绑定区间上界从目标对象

(4) 其他实用方法

- `tooltip(text: str)`: 为组件添加悬停提示
- `delete()`: 彻底删除组件
- `mark(*markers)`: 添加标记 (用于测试或元素查询)

五、使用场景与注意事项

1. 适用场景

- 数据筛选：如电商平台的价格范围筛选、短视频平台的时长筛选、数据报表的时间范围筛选。
- 参数配置：如系统设置中的音量区间、亮度区间、权限等级区间（如 0-50 为普通用户，50-100 为管理员）。
- 阈值划定：如报警系统的温度阈值区间、流量监控的带宽使用区间。

2. 关键注意事项

- 区间约束：`value` 必须是 `[start, end]` 格式的二元列表，且满足 `min ≤ start ≤ end ≤ max`，超出约束时组件会自动修正（如 `start > end` 时交换两者，`start < min` 时设为 `min`）。
- 步长逻辑：`step=0` 表示连续数值（无固定步长），适合需要精准区间的场景（如价格筛选到小数点后两位）；`step>0` 表示离散数值，适合固定增量的场景（如年龄区间以 5 为步长）。
- 事件触发差异：`on_change` 仅在滑块释放时触发，`update:model-value` 在拖动过程中实时触发，需根据场景选择（如实时筛选用后者，确认筛选用前者）。
- 样式作用范围：`classes` 和 `style` 作用于组件容器，滑块、轨道等细节样式需通过 Quasar CSS 变量定制（如 `--q-range-thumb-color`）。
- 版本兼容性：`html_id` 属性仅在 2.16.0+ 版本支持，`bind_*` 方法的 `strict` 参数在 3.0.0+ 版本支持，使用时需确认 NiceGUI 版本。
- 与 `ui.slider` 的区别：
 - 选单个数值 → 用 `ui.slider`；
 - 选数值区间 → 用 `ui.range`；
 - 两者参数和方法高度相似，但 `ui.range` 的 `value` 为二元列表，`ui.slider` 为单个值。

六、进阶示例：带实时筛选的商品价格区间选择

结合数据列表，实现拖动区间滑块时实时筛选商品，模拟电商平台价格筛选功能：

```
from nicegui import ui

# 模拟商品数据（包含名称和价格）
products = [
    {'name': '商品A', 'price': 39},
    {'name': '商品B', 'price': 79},
    {'name': '商品C', 'price': 129},
    {'name': '商品D', 'price': 59},
    {'name': '商品E', 'price': 99},
    {'name': '商品F', 'price': 159},
]

# 区间选择器（0-200 元，步长 1，实时监听）
price_range = ui.range(
    min=0, max=200,
    step=1,
    value=[30, 130],
    label='价格筛选',
```



```

        hint='拖动滑块选择价格范围'
    )

    # 商品列表容器
    product_list = ui.column().classes('mt-4 gap-2')

    # 筛选并更新商品列表的函数
    def filter_products(range_val):
        start, end = range_val
        # 筛选价格在区间内的商品
        filtered = [p for p in products if start ≤ p['price'] ≤ end]
        # 清空列表并重新添加商品
        product_list.clear()
        if filtered:
            for p in filtered:
                ui.label(f'{p["name"]} - {p["price"]} 元').classes('p-2 border rounded')
        else:
            ui.label('暂无符合条件的商品').classes('text-gray-500')

    # 初始加载筛选结果
    filter_products(price_range.value)

    # 实时监听区间变化，更新筛选结果（节流 0.3 秒）
    price_range.on(
        'update:model-value',
        lambda e: filter_products(e.args[0]),
        throttle=0.3
    )

    ui.run()

```

效果：

- 页面加载时，显示价格在 30-130 元之间的商品；
- 拖动滑块调整价格区间，商品列表实时更新（每 0.3 秒触发一次筛选，避免高频渲染）；
- 区间内无商品时，显示“暂无符合条件的商品”提示。